ЛАБОРАТОРНА РОБОТА № 4

«Робота з функціями у JavaScript»

Навчальна дисципліна: WEB-програмування
Варіант: 2
Виконав: студент групи КН-310 Мельник Назар
Місто і рік: Львів — 2026

1. Мета роботи

Ознайомитися з принципами роботи та типами функцій у JavaScript, включаючи іменовані функції, функціональні вирази, стрілкові функції, замикання, рекурсію та колбеки.

2. Короткі теоретичні відомості

Функція — блок коду, який можна повторно викликати та передавати як значення. У JavaScript функції оголошують декларацією `function`, функціональним виразом або стрілковим синтаксисом. Функція вищого порядку приймає іншу функцію чи повертає її. Замикання зберігає доступ до лексичного оточення, а рекурсивна функція викликає сама себе до досягнення базового випадку.

3. Завдання

За методичкою непарні номери журналу виконують варіант 1, парні — варіант 2. Номер студента 14 є парним, тому виконано **варіант 2**. Він містить шість основних завдань і додаткове завдання 7*.

№	Функція	Очікуваний результат
1	`findMax(a, b)`	максимальне з двох чисел
2	`subtract(a, b)`	функціональний вираз, що повертає `a - b`

№	Функція	Очікуваний результат
3	`sqrt(n)`	стрілкова функція квадратного кореня
4	`geometricProgression(n, a, r)`	рекурсивна сума перших `n` членів геометричної прогресії
5	`createDivider(divisor)`	замикання, яке ділить число на збережений дільник
6	`applyOperation(a, b, func)`	застосування переданого колбека; приклади додавання і множення
7*	`processSet(set, callback)`	застосування колбека до всіх елементів `Set`

4. Хід роботи

У готовому проєкті знайдено `index.html` і `main.js`. HTML підключає скрипт через `<script src="main.js" defer></script>`, тому код виконується після розбору сторінки. Код усіх семи завдань звірено з методичкою. Алгоритмічних або синтаксичних помилок не виявлено, тому вихідний код не змінювався.

4.1. Іменована, функціональний вираз і стрілкова функція

```javascript
function findMax(a, b) {
return a > b ? a : b;
}

const subtract = function(a, b) {
return a - b;
};

const sqrt = n => Math.sqrt(n);
```

`findMax` використовує тернарний оператор, `subtract` реалізовано саме як функціональний вираз, а `sqrt` — як стрілкову функцію.

4.2. Рекурсивна геометрична прогресія

```
javascript function geometricProgression(n, a, r) { if (n === 0) { return 0; } return a + r * geometricProgression(n - 1, a, r); }
```

Базовий випадок для нульової кількості членів повертає 0. Рекурентна формула еквівалентна сумі `a + ar + ar² + ...`. Для `n=4`, `a=2`, `r=3` отримано `2 + 6 + 18 + 54 = 80`.

4.3. Замикання і колбеки

```javascript
function createDivider(divisor) {
return function(number) {
return number / divisor;
};
}

function applyOperation(a, b, func) {
return func(a, b);
}

function processSet(set, callback) {
set.forEach(callback);
}
```

Внутрішня функція `createDivider` зберігає значення `divisor`. Функції `applyOperation` і `processSet` викликають передані колбеки.

5. Фактичні результати

5.1. Node.js та алгоритмічні перевірки

Виконано:

```
bash node --check main.js node main.js node /tmp/web_lr04_algorithm_tests.js
```

Синтаксична перевірка успішна. Автоматично пройдено **19 перевірок**: приклади методички, переставлені й рівні аргументи `findMax`, від'ємні числа, нульовий і від'ємний аргументи `sqrt`, базові випадки рекурсії, коефіцієнт `r=1`, замикання з різними дільниками, обидва колбеки, унікальність `Set` і точний консольний вивід.

Перевірка	Вхід	Результат
максимум	`12, 7`	12
віднімання	`12, 7`	5
корінь	`81`	9
прогресія	`4, 2, 3`	80
замикання	дільник 2, число 20	10
додавання / множення	`5, 3`	8 / 15
`Set([1,2,3])`	подвоєння	2, 4, 6

5.2. Перевірка сторінки Playwright

Playwright відкрив сторінку через локальний HTTP-сервер і підтвердив:

- HTTP 200 для `/` та `/main.js`;
- правильний заголовок сторінки й наявність підключеного `main.js`;
- 11 точних console-повідомлень із результатами завдань 1–7*;
- `page_errors=0` і `request_failures=0`.

Лабораторна робота №4

Робота з функціями у JavaScript. Варіант 2.

Відкрийте інструменти розробника (F12) → Console, щоб побачити результати.

Рисунок 1 — Запущена сторінка WEB ЛР №4.

5.3. Результати JavaScript

Нижче наведено справжній знімок окремого вікна Konsole після виконання `node main.js`. Він не є імітацією браузерної консолі; браузерні console-події окремо записано Playwright у `logs/04_playwright.txt`.

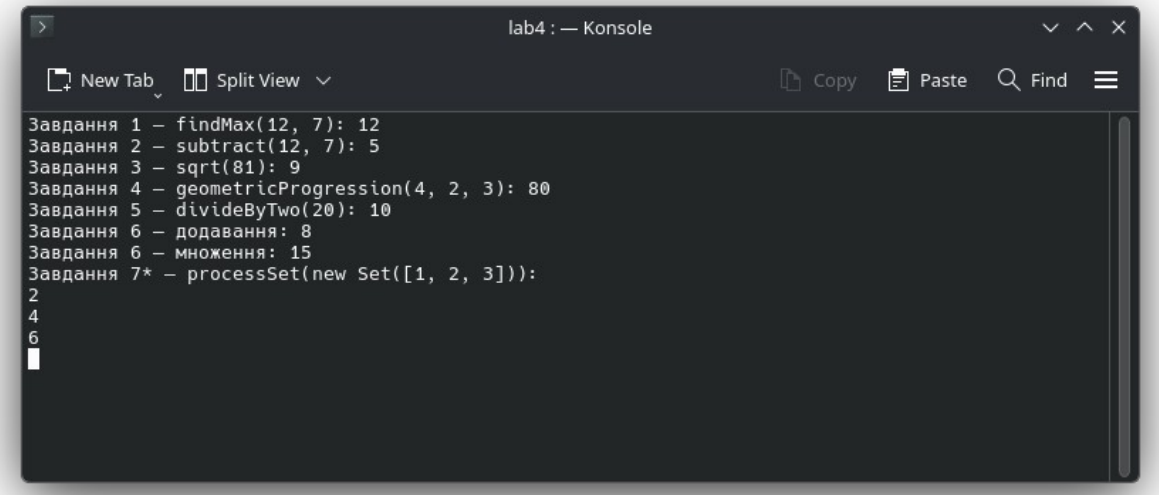


Рисунок 2 — Фактичні результати виконання семи JavaScript-завдань.

6. Відповіді на питання вихідного контролю

6.1. Які існують способи оголошення функцій у JavaScript?

Декларація `function name() {}`, функціональний вираз `const f = function() {}` та стрілкова функція `const f = () => {}`.

6.2. Що таке функціональний вираз і чим він відрізняється від декларації?

Функціональний вираз створює функцію як значення виразу й присвоює її змінній. Декларація функції доступна раніше за місце оголошення завдяки hoisting; змінну `const` із функціональним виразом можна використовувати лише після ініціалізації.

6.3. Як працює замикання?

Внутрішня функція зберігає доступ до змінних зовнішньої лексичної області після завершення зовнішньої функції. У `createDivider` зберігається `divisor`.

6.4. Що таке колбек-функція?

Це функція, передана іншій функції як аргумент для подальшого виклику. Приклади з роботи: операції, передані `applyOperation`, і callback у `Set.forEach` всередині `processSet`.

6.5. Як реалізувати рекурсію у JavaScript?

Функція викликає сама себе зі спрощеним аргументом і обов'язково має базовий випадок. У `geometricProgression` ним є `n === 0`.

6.6. Як працює оператор rest `...` при оголошенні функції?

Останній параметр із `...` збирає решту аргументів у масив, наприклад `function f(first, ...rest) {}`.

6.7. Як працює spread `...` під час виклику?

Spread розгортає ітерований об'єкт в окремі аргументи. Наприклад,

`Math.max(...[2, 3])` еквівалентний `Math.max(2, 3)`.

7. Обмеження

Git-гілку `lab4`, посилання в меню `labs/index.html`, merge, push і Netlify не підтверджено: окремого навчального репозиторію `labs` та даних доступу до публікації в наявних матеріалах немає. Відповідно до умов перевірки Git push і публікація не виконувались.

8. Висновки

Перевірено всі сім завдань варіанта 2. Код коректно демонструє декларацію, функціональний вираз, стрілкову функцію, рекурсію, замикання, функцію вищого порядку та callback для `Set`. Синтаксис, алгоритми, точні приклади та браузерне виконання підтверджено реальними автоматичними запусками. Помилок у `main.js` не знайдено, тому правильний вихідний код залишено без змін.